

HAPPY DONUT

Expediente final del producto software

Arquitectura de microservicios, DDD, calidad, GitOps, SRE y resiliencia

5 bounded contexts	21 paths OpenAPI	7 imágenes locales	1 instalador reproducible
-----------------------	---------------------	-----------------------	------------------------------

Recurso	Enlace
Repositorio	github.com/happy-donut-software/Happy-Donnut-Software
Aplicación local	http://localhost:30080
Administración	http://localhost:30080/admin/
Argo CD	http://localhost:30081
Grafana	http://localhost:30300

Entorno objetivo: Docker Desktop Kubernetes local, sin servidor de pago. Las URL de paneles se habilitan al ejecutar `scripts/instalar-local.ps1`.

Fecha de corte: 15 de julio de 2026 · Rama de integración: develop

1. Objetivo y alcance

Convertir los requisitos obtenidos en la entrevista de Kelly Flores en un producto ejecutable, trazable y operable. El alcance implementado cubre venta presencial y web, cobro EFECTIVO/YAPE/PLIN, vuelto, boleta/nota de pedido, actualización automática de inventario, alerta de stock mínimo, caja, acumulado RUS, clientes frecuentes, observabilidad, GitOps y resiliencia local.

2. DDD estratégico y trazabilidad

`docs/ddd/lenguaje-ubicuo.md` define el vocabulario del negocio. `event-storming.md` registra comandos, eventos, políticas, actores y puntos críticos. `context-map.md` explica las relaciones Customer/Supplier y Published Language entre Ventas, Inventario y Finanzas. La matriz `docs/trazabilidad.md` enlaza cada RF/RNF con frase fuente, modelo, escenario BDD, endpoint, métrica y archivo.

Los bounded contexts conservan bases independientes. `VentaFinalizada.v1` es el contrato publicado; su identificador de evento permite entrega al menos una vez sin duplicar efectos.

3. Diseño táctico y arquitectura hexagonal

Los agregados principales son `OrdenVenta`, `ProductoInventario`, `TurnoCaja`, `AcumuladoRus`, `Usuario` y `CarritoCompras`. Entidades, value objects y reglas permanecen en Dominio; casos de uso y DTO en Aplicación; Eloquent, HTTP y Laravel en Infraestructura.

El pago usa transaccional outbox: el estado de `OrdenVenta` y el evento se confirman en una transacción. `ventas:publicar-eventos` actúa como relay con timeout y reintentos. Inventario y Finanzas registran `evento_id` dentro de su propia transacción, por lo que una repetición responde como duplicada y no vuelve a descontar ni acumular.

4. Funcionalidad verificable

El panel administrativo implementa selección de productos, cliente frecuente opcional, carrito, medio de pago, comprobante, importe recibido, vuelto y confirmación. El portal cliente consume catálogo real, registro/login Sanctum y compra. Los seeders crean catálogo, stock, cajero, administrador, clientes frecuentes y una caja local abierta.

`qa-e2e/steps.cjs` ejecuta contra el gateway un recorrido real: consulta stock y RUS, crea/paga una venta, espera el relay y verifica decremento de stock e incremento fiscal.

5. Calidad

`features/` contiene especificaciones Gherkin y `features/e2e_local.feature` es ejecutable con Cucumber. PHPUnit cubre reglas de orden, vuelto, outbox, inventario, caja y RUS. CI exige cobertura mínima de 80 % mediante PCOV. Infection exige MSI 70 % y covered MSI 80 %. Los informes XML de cobertura y mutación se publican por el pipeline; el criterio no se declara cumplido hasta una ejecución verde del SHA final.

El contrato OpenAPI 3.1 enumera los 21 paths HTTP reales, incluyendo integración y métricas. Redocly lo valida en CI.

6. SCM y GitOps

El plan `docs/scm/plan-gestion-configuracion.md` sigue IEEE 828: identifica CIs, responsables, líneas base, versionado, auditoría, cambio y liberación. GitHub Actions verifica cada PR y las imágenes de producción se etiquetan por SHA.

Argo CD usa `prune` y `selfHeal`. El manifiesto local observa `develop`; el productivo observa `main`. La activación local exige `commit/push` para evitar confundir archivos del disco con estado GitOps.

7. SRE, telemetría y caos

Prometheus recoge `http_requests_total` y el histograma `http_request_duration_seconds`. Las reglas alertan por burn rate, p95 superior a 500 ms y deployments sin réplica. Grafana muestra disponibilidad, latencia y presupuesto consumido. OpenTelemetry PHP auto-instrumenta Laravel/PDO, el Collector recibe OTLP y Tempo persiste trazas.

`docs/sre/SLI-SLO.md` define SLI/SLO y presupuesto. `chaos/` incluye caída de pod y latencia. Los experimentos solo deben ejecutarse localmente o en staging, observando recuperación y presupuesto; nunca se presentan resultados simulados.

8. Despliegue y evidencia

`scripts/instalar-local.ps1` instala versiones fijadas de Argo CD, kube-prometheus-stack, Tempo, OpenTelemetry Collector y Chaos Mesh; construye las imágenes; aplica Kustomize; migra y siembra; carga dashboards; y ejecuta Cucumber. `scripts/verificar-local.ps1` produce un ZIP fechado con evidencia técnica y commit.

9. Conclusión

El repositorio contiene una ruta local completa desde requisito hasta dominio, API, interfaces, persistencia, eventos, despliegue, telemetría y evidencia. Para exposición pública faltaría infraestructura externa - DNS/TLS, secretos gestionados, registro y bases administradas -, lo cual queda deliberadamente separado del entorno académico local.

Referencias

Evans, E. (2003). **Domain-Driven Design**. Addison-Wesley. Vernon, V. (2013). **Implementing Domain-Driven Design**. Addison-Wesley. IEEE Std 828-2012. **Configuration Management in Systems and Software Engineering**. ISO/IEC/IEEE 29119. **Software Testing**. Beyer, B. et al. (2016). **Site Reliability Engineering**. O'Reilly.